

Holberton Portfolio Project - Technical Documentation (Stage 3)

Objectives:

- Define User Stories and Mockups
- Design System Architecture
- Define Components, Classes, and Database Design
- Create High-Level Sequence Diagrams
- Document External and Internal APIs
- Plan SCM and QA Strategies

Deliverables:

- User Stories, Wireframes and Mockups.
- High-level diagram.
- ER diagram, database schema, or collection structure.
- Sequence Diagrams.
- External APIs and internal API endpoints.
- Strategies for source control and testing.
- Technical Justifications.

User Stories:

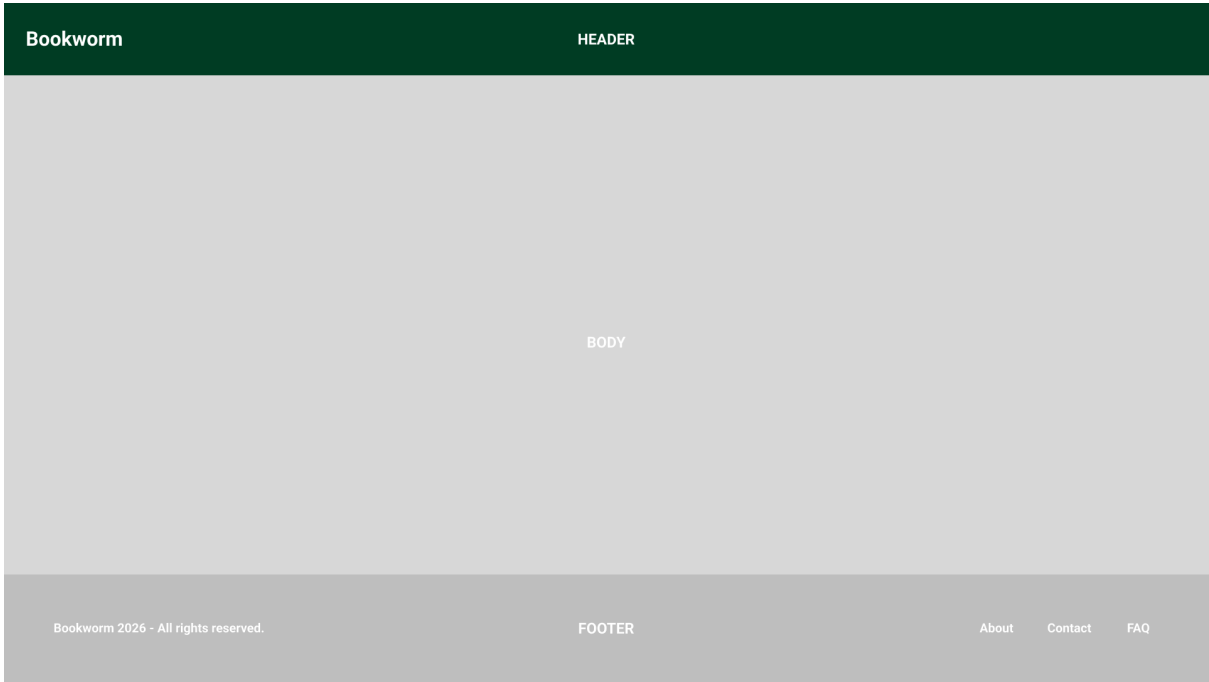
User	Story
Business Owner	“As a Restaurant Owner I want to be able to book client reservations to create a work schedule”.
Business Employee	“As a Hostess I want to be able to quickly check available tables for new coming clients”.
Business Client	“As a Client I want to be able to get to a place and be served by just giving my name and reservation date”.
Business Owner	“As a Doctor I want to be able to schedule user appointments to know how much time to assign to each appointment”.

Interface Mockups:

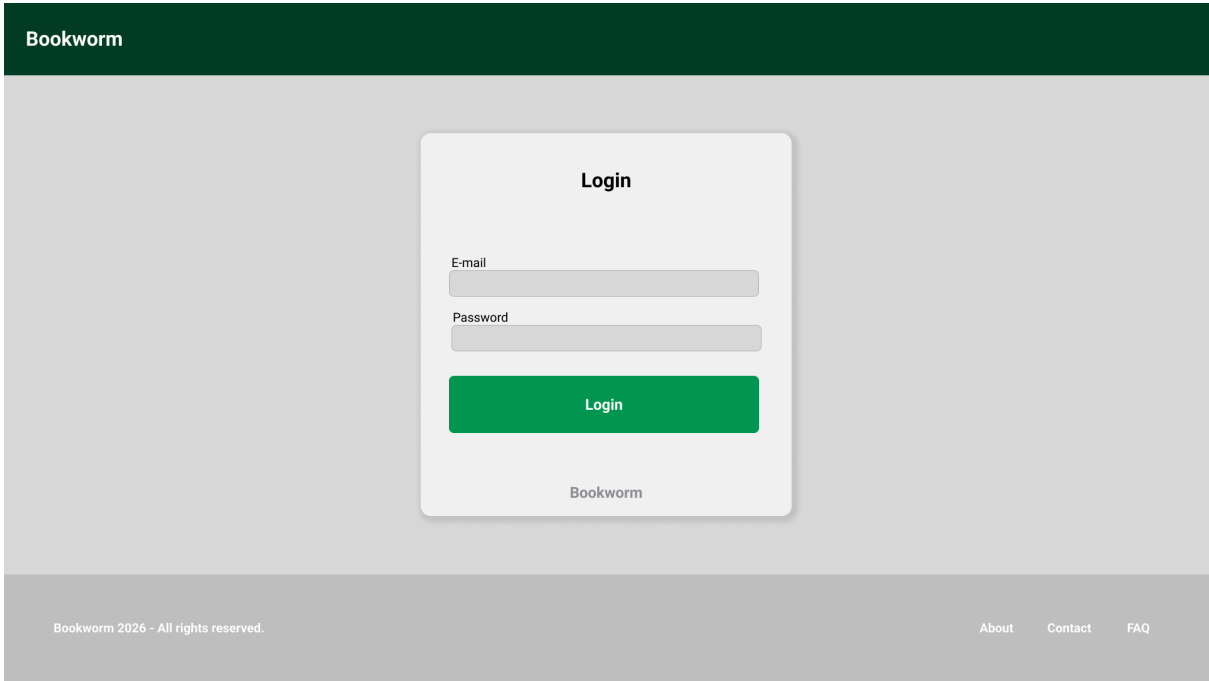
CSS Framework: Tailwind CSS

Icons: HeroIcons

Base Layout



Login Form



Register Form

Bookworm

Register

First Name

Last Name

E-mail

Password

Confirm Password

Login

Bookworm

Bookworm 2026 - All rights reserved.

About

Contact

FAQ

Dashboard:

Bookworm

Locations

Reservations

Users

Log Out

Locations

Add

Filter

Bookworm 2026 - All rights reserved.

About

Contact

FAQ

Add Establishment (Location) Form

Bookworm

Locations

Reservations

Users

Log Out

Add Location

New Location

Name of the establishment

Capacity (Tables, slots, etc)

Description (optional)

Login

Bookworm 2026 - All rights reserved.

About

Contact

FAQ

Add Reservation Form

Bookworm

Locations

Reservations

Users

Log Out

Add Reservation

New Reservation

Client Name

Date

Reason

Slot number (Table, office, etc)

Add reservation

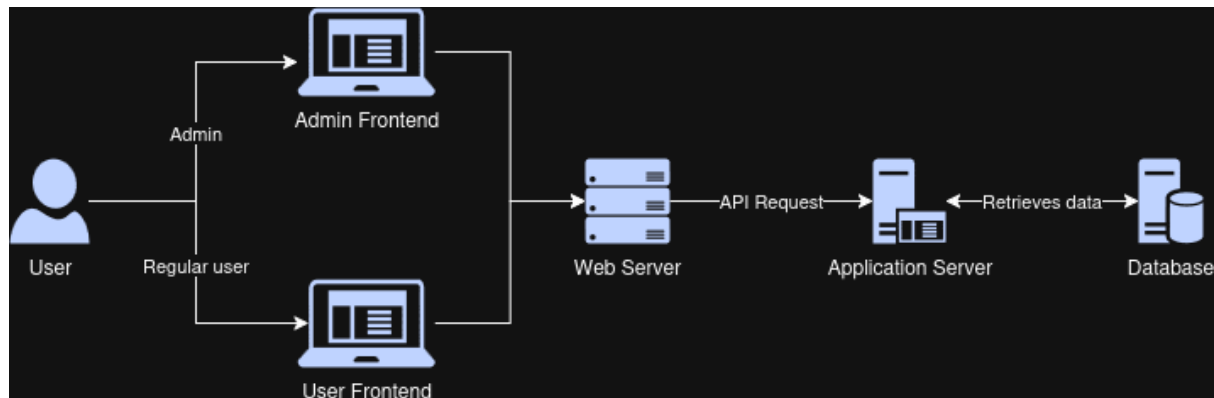
Bookworm 2026 - All rights reserved.

About

Contact

FAQ

High Level Architecture Diagram



Internal API

All API endpoints will follow the naming convention of “api/version_number”. For example: “api/v1” defines the route for the version 1 of the api. This approach allows us to progressively modify our core application without completely changing how different elements of the applications interact with each other (for example, how our frontend makes API calls).

SCM Processes:

Version Control: Github

Branching Strategy: We will implement a simple branching schema for our application. We will have two main branches: master and develop. The master branch will be the main system version and will be used for the production environment. The develop branch will be used for testing and adding updates to the app. For any major feature added a branch will be created from develop and merge back when the feature is stable. For critical situations a hotfix branch can be created from master to quickly fix crucial errors in production.

- **Main branch:** Production environment and main versioning.
- **Develop branch:** Testing environment and adding updates/fixes.
- **Feature branch:** Adding extra features to the application.
- **Hotfix branch:** Quickfix any critical error in production.

QA Processes: For reviewing API endpoints and features we will mainly use two tools: Python’s unittest module for automated tests and Postman for manual testing. For our deployment we will use Render as it automatically updates the system when we push to a specified branch of our repository.

- **Unit tests:** Python Unittest.
- **Manual testing:** Postman.
- **Deployment and CI/CD:** Render